

GitHub na Prática

O que é GitHub? A analogia da nuvem de código

Imagine que você está escrevendo um livro. Você salva o arquivo no seu computador — mas e se o HD queimar? E se quiser trabalhar em um café diferente? E se um colega precisar revisar o capítulo 3 ao mesmo tempo que você escreve o capítulo 5?

Git é o sistema que registra cada versão do seu texto, como se tirasse uma **foto do projeto** a cada mudança importante.

GitHub é a **nuvem** onde essas fotos ficam guardadas — acessível de qualquer computador, podendo ser compartilhada com qualquer pessoa.

 Git ≠ GitHub. Git é o motor. GitHub é a garagem na nuvem onde você estaciona o projeto.

Dicionário visual do GitHub

Termo	Analogia	O que significa
Repositório	A pasta do projeto	Um "projeto" no GitHub com todos os arquivos e histórico
Commit	Uma foto com legenda	Um ponto salvo no tempo com uma mensagem explicando o que mudou
Branch	Uma rascunho paralelo	Uma versão alternativa onde você experimenta sem afetar o original
Clone	Baixar o projeto	Copia o repositório da nuvem para o seu computador
Push	Enviar para a nuvem	Manda suas alterações do computador para o GitHub
Pull	Buscar da nuvem	Traz as alterações mais recentes da nuvem para o seu computador
main / master	O tronco da árvore	A branch principal — a versão "oficial" do projeto

Instalando as ferramentas

O que você vai instalar:

- **Git (o motor)** — baixado em segundo plano pelo GitHub Desktop
- **GitHub Desktop** — interface visual, sem precisar digitar comandos

Passo a passo — siga na ordem:

1

Criar conta no GitHub (gratuita)

Acesse github.com → clique em "Sign up" → use o e-mail que você usa sempre

2

Baixar o GitHub Desktop

Acesse desktop.github.com → clique em "Download for Windows" (ou Mac)

3

Instalar e fazer login

Abra o instalador → siga o assistente → entre com sua conta GitHub quando pedir

4

Confirmar que funcionou

O GitHub Desktop vai abrir mostrando sua tela inicial. Pronto!

Criando o seu primeiro repositório

Repositório é o "projeto" no GitHub. Vamos criar um para o projeto InovaWeb do curso.

Opção A — Criar pelo site github.com:

1. Acesse github.com e faça login
2. Clique no botão verde "New" (ou no "+" no canto superior direito → "New repository")
3. Preencha os campos:

```
Repository name: inovaweb-frontend
Description:      Projeto do curso Front-End — InovaWeb
Visibilidade:    Public (ou Private se preferir)
☒ Marque:      Add a README file
```

4. Clique em "Create repository"

Opção B — Criar direto no GitHub Desktop:

5. Abra o GitHub Desktop
6. Clique em "File" → "New Repository"
7. Preencha o nome, descrição e escolha onde salvar no seu computador
8. Marque "Initialize this repository with a README"
9. Clique em "Create Repository"
10. Clique em "Publish repository" para enviar ao GitHub

Clonando — trazendo o projeto para o seu computador

Clonar = baixar uma cópia do repositório da nuvem para o seu computador. Você faz isso uma única vez por projeto.

No GitHub Desktop:

11. Abra o GitHub Desktop
12. Clique em "File" → "Clone repository"
13. Escolha a aba "GitHub.com" — seus repositórios aparecem automaticamente
14. Clique no repositório desejado (ex: inovaweb-frontend)
15. Escolha onde salvar no seu computador (ex: Documentos/projetos)
16. Clique em "Clone"

 Após clonar, o GitHub Desktop mostra o repositório na lista à esquerda. A pasta local aparece no Windows Explorer normalmente.

Visualizando a pasta criada:

No GitHub Desktop, clique em **"Repository"** → **"Show in Explorer"** para abrir a pasta. Você verá o arquivo `README.md` lá dentro.

Fazendo o primeiro Commit — a "foto" do projeto

Commit é o momento em que você diz: "Está bom assim, quero guardar esse ponto." É como salvar o jogo.

Fluxo completo — do arquivo ao GitHub:

1

Edite um arquivo

Abra o projeto no VSCode e salve alguma alteração (ex: edite o README.md)

2

Veja as mudanças no GitHub Desktop

Volte ao GitHub Desktop — ele detecta automaticamente os arquivos alterados e lista na seção "Changes"

3

Escreva uma mensagem de commit

No campo "Summary", escreva uma frase curta descrevendo o que mudou

4

Clique em "Commit to main"

A foto foi tirada! Está salva no seu computador, mas ainda não na nuvem

5

Clique em "Push origin"

Agora sim — o commit foi para o GitHub! Abra github.com e veja

 Dica de ouro: escreva mensagens de commit no imperativo e em português. "Adiciona formulário de contato" é melhor que "mudancas".

Exemplos de boas mensagens de commit:

- ☒ Adiciona página inicial da InovaWeb
- ☒ Corrige layout do menu de navegação
- ☒ Atualiza cores conforme identidade visual
- ☒ atualizei coisas
- ☒ teste
- ☒ aaaa

Push e Pull — sincronizando com a nuvem

Push = enviar seus commits do computador para o GitHub (você → nuvem).

Pull = trazer commits do GitHub para o seu computador (nuvem → você).

 Push	 Pull
Você trabalhou no computador e quer salvar na nuvem	Alguém atualizou o projeto na nuvem e você quer trazer
GitHub Desktop: clique em "Push origin"	GitHub Desktop: clique em "Fetch origin" → "Pull origin"

Ao trabalhar em equipe, SEMPRE faça Pull antes de começar a trabalhar. Assim você começa com a versão mais atualizada.

Vendo o histórico — o álbum de fotos do projeto

O histórico mostra todas as "fotos" (commits) tiradas ao longo do tempo. É possível ver o que mudou em cada uma.

No GitHub Desktop:

- **"History"** (ao lado de "Changes")
- Cada linha é um commit — com data, autor e mensagem
- Clique em qualquer commit para ver exatamente o que foi alterado (linhas vermelhas = removidas, verdes = adicionadas)

No github.com:

- Acesse seu repositório
- **"X commits"** (aparece acima dos arquivos)
- Você vê o histórico completo com todos os detalhes

Se você "quebrar" algo no projeto, o histórico te permite ver quando o erro foi introduzido e o que mudou. É sua rede de segurança.

Branches — trabalhando sem medo de quebrar

Branch é como fazer uma cópia do projeto para experimentar algo novo — sem alterar a versão principal. Se der certo, você une as duas. Se não, descarta sem dor.

A analogia do caderno:

Imagine que seu projeto é um caderno de receitas. A branch main é o caderno original. Você quer testar uma receita nova mas tem medo de estragar o original — então você pega uma folha avulsa (branch). Se a receita ficar boa, você cola no caderno. Se não, amassa e joga fora.

Criando uma branch no GitHub Desktop:

17. Clique no seletor de branch no topo central (onde está escrito "main")

18. Clique em "New branch"

19. Dê um nome descritivo: ex:

```
feature/pagina-contato  
fix/menu-responsivo  
test/nova-paleta-cores
```

20. Clique em "Create branch"

21. A partir de agora, seus commits vão para essa branch, não para a main

Use nomes de branch que descrevem o que está sendo feito. "minha-branch" não diz nada. "feature/formulario-newsletter" diz tudo.

Voltando para a main:

Clique no seletor de branch → escolha "main". O GitHub Desktop troca a branch automaticamente e os arquivos da pasta refletem a versão correta.

README.md — o cartão de visitas do projeto

O arquivo `README.md` é a primeira coisa que qualquer pessoa vê ao acessar seu repositório no GitHub. Ele usa uma linguagem chamada **Markdown** — simples de escrever, bonita de ler.

Exemplo de README básico para o projeto InovaWeb:

```
# InovaWeb — Frontend

Projeto do curso de Programador Front-End — Senac RJ.

## Tecnologias usadas
- HTML5
- CSS3
- JavaScript

## Como abrir
1. Clone o repositório
2. Abra o arquivo index.html no navegador

## Autores
- Seu Nome — github.com/seu-usuario
```

Sintaxe básica do Markdown:

Termo	Analogia	O que significa
# Título	Título de página	Quanto mais #, menor o título (# maior, ### menor)
negrito	Texto em negrito	Envolva o texto com dois asteriscos
<i>*itálico*</i>	Texto em itálico	Envolva o texto com um asterisco
- item	Lista com bullet	Hífen + espaço no início da linha
1. item	Lista numerada	Número + ponto + espaço
[link] (url)	Link clicável	Texto entre [] e URL entre ()

Fluxo completo do dia a dia

Este é o ciclo que você vai repetir em todos os seus projetos:

1	 Pull	Inicie sempre buscando a versão mais atualizada da nuvem
2	 Edite	Faça suas alterações no VSCode normalmente
3	 Commit	No GitHub Desktop: escreva mensagem → "Commit to main"
4	 Push	Clique em "Push origin" — seus commits vão para o GitHub
5	 Repita	Continue: Pull → Edit → Commit → Push quantas vezes precisar

 Commite com frequência! Não espere o projeto estar "perfeito" — commit frequente = histórico rico = maior segurança.

Erros comuns e como resolver

Situação	Problema	Solução
Esqueci de fazer Pull antes de editar	Conflito de versões com a nuvem	No GitHub Desktop, clique em "Fetch origin" e depois "Pull origin" antes de começar
Fiz commit mas não subi para o GitHub	Outros não veem suas mudanças	Clique em "Push origin" no GitHub Desktop
Mensagem de commit muito vaga	Impossível entender o histórico depois	Sempre escreva o QUE foi feito: "Adiciona botão de contato"
Adicionei arquivos sensíveis (senha, API key)	Dado privado exposto publicamente	Crie um arquivo .gitignore com os nomes dos arquivos a ignorar
"Not authenticated" ao fazer push	Não está logado ou token expirou	No GitHub Desktop: File → Options → Accounts → Sign In

.gitignore — o que NÃO vai para o GitHub

O arquivo `.gitignore` lista arquivos e pastas que o Git deve **ignorar** — ou seja, nunca enviar para o GitHub. É essencial para não expor senhas e não poluir o repositório.

Exemplo de .gitignore para projetos front-end:

```
# Dependências (geradas automaticamente, são enormes)
node_modules/

# Arquivos de configuração local e senhas — NUNCA suba isso!
.env
.env.local

# Arquivos do sistema operacional
.DS_Store      # Mac
Thumbs.db      # Windows

# Pastas de build geradas
dist/
build/
```

🌐 Dica rápida: o site gitignore.io gera um `.gitignore` personalizado para o seu tipo de projeto. Digite "HTML CSS JavaScript" e ele cria o arquivo pra você.

Dicas de ouro para o dia a dia

- **Commite pequeno e frequente:** um commit = uma mudança lógica, não um semana inteira de trabalho
- **Mensagem no imperativo:** "Adiciona" e não "Adicionei" — é uma convenção do mercado
- **Pull antes de tudo:** antes de abrir o VSCode, já faça Pull no GitHub Desktop
- **Seu perfil é seu portfólio:** recrutadores de TI verificam o GitHub. Projetos públicos valem ouro
- **Repositório com README:** todo projeto público deve ter README. Mostra profissionalismo
- **Use branches para experimentos:** nunca experimente direto na main em projetos reais

Glossário rápido — para colar na parede:

<code>git</code>	Sistema de controle de versão
<code>GitHub</code>	Plataforma na nuvem para hospedar repositórios Git
<code>repositório (repo)</code>	Pasta do projeto com histórico de alterações
<code>commit</code>	Ponto salvo no tempo com mensagem
<code>branch</code>	Linha de desenvolvimento paralela
<code>main / master</code>	Branch principal do projeto
<code>clone</code>	Cópia local de um repositório remoto
<code>push</code>	Enviar commits para o repositório remoto
<code>pull</code>	Buscar atualizações do repositório remoto
<code>merge</code>	Unir uma branch com outra
<code>README.md</code>	Arquivo de apresentação do projeto (Markdown)
<code>.gitignore</code>	Arquivo com lista de itens que o Git deve ignorar